

Running the RTOS on a ARM Cortex-M Core

KernelARM Cortex

Updated May 2025

[see also [debugging Cortex hard fault exceptions](#)].

Note: The information regarding interrupt nesting on this page applies when using a Cortex-M3, Cortex-M4, Cortex-M4F, Cortex-M7, Cortex-M33 and Cortex-M23. It does not apply to Cortex-M0 or Cortex-M0+ cores, which do not include a BASEPRI register.

Introduction

Many thousands of applications run FreeRTOS on ARM Cortex-M cores. It is surprising, then, that there are so few technical support requests for this RTOS and ARM Cortex CPU core combination. The majority of issues that do occur are the result of incorrect interrupt priority settings. This is probably to be expected because, although the interrupt model used by the ARM Cortex-M core is powerful, it is also somewhat clumsy and counter intuitive to engineers who are used to a more conventional interrupt priority scheme. This page aims to describe the ARM Cortex-M interrupt priority mechanism, and describe how it should be used with the RTOS kernel.

Remember that, although the priority scheme imposed by the ARM Cortex-M3 core may seem complex, each official FreeRTOS port comes with a correctly configured demo application that can be used as a reference. In addition FreeRTOS V7.5.0 introduced additional configASSERT() calls

specifically to catch mis-configured ARM Cortex-M interrupt controllers (NVIC). **Please ensure [configASSERT\(\) is defined](#) during development.**

Available Priority Levels

Cortex-M hardware details

The first thing to know is that the total number of available priorities is implementation defined, that is, up to the manufacturer of the microcontroller that uses the ARM Cortex-M core. As a result, not all ARM Cortex-M microcontrollers provide the same number of unique interrupt priorities.

The ARM Cortex-M architecture itself allows a maximum of 256 different priorities (there are a maximum of eight priority bits, so priorities 0 to 0xff inclusive are possible), but most, if not all, microcontrollers that incorporate a ARM Cortex-M core only allow access to a subset of these. For example, the TI Stellaris Cortex-M3 and ARM Cortex-M4 microcontrollers implement three priority bits. This provides for eight unique priority values. As another example, the NXP LPC17xx ARM Cortex-M3 microcontrollers implement five priority bits. This provides for 32 unique priority values.

If your project includes CMSIS library header files, then inspect the `__NVIC_PRIO_BITS` definition to see how many priority bits are available.

Relevance when using the RTOS

The RTOS interrupt nesting scheme splits the available interrupt priorities into two groups - those that will get masked by RTOS critical sections, and those that are never masked by RTOS critical sections and are therefore always enabled. The [configMAX_SYSCALL_INTERRUPT_PRIORITY](#) setting in

FreeRTOSConfig.h defines the boundary between the two groups. The optimal value for this setting will depend on the number of priority bits implemented in the microcontroller.

Preempt Priority and Subpriority

Cortex-M hardware details

The 8-bit priority register is divided into two parts: preempt priority and sub priority. The number of bits assigned to each part is configurable. The preempt priority defines whether an interrupt can preempt an already executing interrupt. The sub priority determines which interrupt will execute first when two interrupts of the same preempt priority occur at the same time.

Relevance when using the RTOS

It is recommended to assign all the priority bits to be preempt priority bits, leaving no priority bits as subpriority bits. Any other configuration complicates the otherwise direct relationship between the configMAX_SYSCALL_INTERRUPT_PRIORITY setting and the priority assigned to individual peripheral interrupts.

Most systems default to the wanted configuration, with the noticeable exception of the STM32 driver library. **If you are using an STM32 with the STM32 driver library then ensure all the priority bits are assigned to be preempt priority bits by calling NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4); before the RTOS is started.**

Inverse Relationship Between Numeric Priority Value and the Logical Priority Setting

Cortex-M hardware details

The next thing to know is that, in ARM Cortex-M cores, numerically low priority values are used to specify logically high interrupt priorities. For example, the logical priority of an interrupt assigned a numeric priority value of 2 is above that of an interrupt assigned a numeric priority of 5. In other words, interrupt priority 2 is higher than interrupt priority 5, even though the number 2 is lower than the number 5. To hopefully make that clear: An interrupt assigned a numeric priority of 2 can interrupt (nest with) an interrupt assigned a numeric priority of 5, but an interrupt assigned a numeric priority of 5 cannot interrupt an interrupt assigned a numeric priority of 2.

This is the most counterintuitive aspect of ARM Cortex-M interrupt priorities because it is the opposite to most non ARM Cortex-M3 microcontroller architectures.

Relevance when using the RTOS

FreeRTOS functions that end in "FromISR" are interrupt safe, but even these functions cannot be called from interrupts that have a logical priority above the priority defined by `configMAX_SYSCALL_INTERRUPT_PRIORITY` ([configMAX_SYSCALL_INTERRUPT_PRIORITY is defined in the FreeRTOSConfig.h header file](#)). Therefore, any interrupt service routine that uses an RTOS API function must have its priority manually set to a value that is numerically equal to or **greater than** the `configMAX_SYSCALL_INTERRUPT_PRIORITY` setting. This ensures the interrupt's logical priority is equal to or less than the `configMAX_SYSCALL_INTERRUPT_PRIORITY` setting.

Cortex-M interrupts default to having a priority value of zero. Zero is the

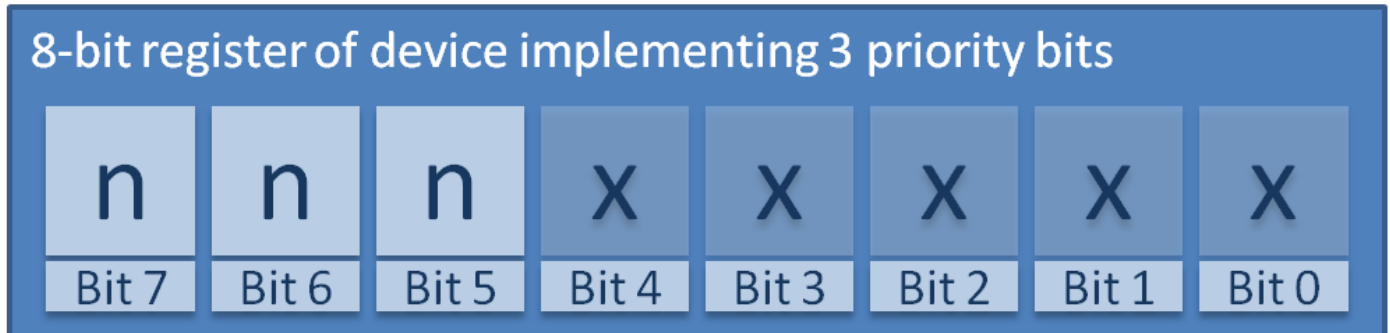
highest possible priority value. Therefore, **never leave the priority of an interrupt that uses the interrupt safe RTOS API at its default value.**

Cortex-M Internal Priority Representation

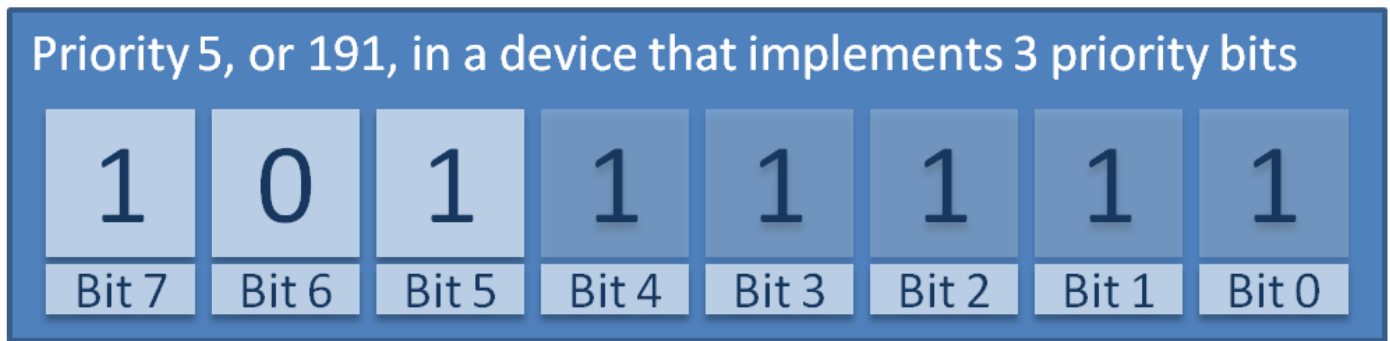
Cortex-M hardware details

The ARM Cortex-M core stores interrupt priority values in the most significant bits of its eight bit interrupt priority registers. For example, if an implementation of a ARM Cortex-M microcontroller only implements three priority bits, then these three bits are shifted up to be bits five, six and seven respectively. Bits zero to four can take any value, although, for future proofing and maximum compatibility, they should be set to one.

The internal ARM Cortex-M representation is demonstrated in the images below.



Cortex-M priority registers have space for a maximum of eight priority bits. If, as an example, a microcontroller only implements three bits, then it is the three most significant bits that are used.



The diagram above shows how the value 5 (binary 101) is stored in a priority register of a microcontroller that implements three priority bits. The diagram demonstrates why the value 5 (binary 0000 0101) can also be considered to be 191 (binary 1011 1111) when the three bits are shifted into the required position and the remaining bits are set to 1.



The diagram above shows how the value 5 (binary 101) is stored in a priority register of a microcontroller that implements four priority bits. The diagram demonstrates why the value 5 (binary 0000 0101) can also be considered to be 95 (binary 0101 1111) when the four bits are shifted into the required position and the remaining bits are set to 1.

Relevance when using the RTOS

As described above, it is essential that interrupt service routines that make use of the RTOS API have a logical priority equal to or below that set by the `configMAX_SYSCALL_INTERRUPT_PRIORITY` (lower logical priority means higher numeric value).

CMSIS, and different microcontroller manufacturers, provide library functions that can be used to set the priority of an interrupt. Some library functions expect the interrupt priority to be specified in the least significant bits of an eight bit bytes, while others expect the interrupt priority to be specified already shifted to the most significant bits of the eight bit byte. Check the documentation for the function being called to see which is required in your case, as getting this wrong can lead to unexpected behaviour.

The `configMAX_SYSCALL_INTERRUPT_PRIORITY` and `configKERNEL_INTERRUPT_PRIORITY` settings found in `FreeRTOSConfig.h` require their priority values to be specified as the ARM Cortex-M core itself wants them - already shifted to the most significant bits of the byte. That is why `configKERNEL_INTERRUPT_PRIORITY`, which should be set to the lowest interrupt priority, is set to 255 (1111 1111 in binary) in the `FreeRTOSConfig.h` header files delivered with each official FreeRTOS demo. The values are specified this way for a number of reasons: The RTOS kernel accesses the ARM Cortex-M3 hardware directly (without going through any third party library function), the RTOS kernel implementation pre-dates most library function implementations, and this was the scheme used by the first ARM Cortex-M3 libraries to come to market.

Critical Sections

Cortex-M hardware details

The RTOS kernel implements critical sections using the ARM Cortex-M core's BASEPRI register. This allows the RTOS kernel to only mask a subset of interrupts, and therefore provide a flexible interrupt nesting model.

BASEPRI is a bit mask. Setting BASEPRI to a value masks all interrupts that

have a priority at and (logically) below that value. It is therefore not possible to use BASEPRI to mask interrupts that have a priority of 0.

An aside: FreeRTOS API functions that are safe to be called from an interrupt use BASEPRI to implement interrupt safe critical sections. BASEPRI is set to configMAX_SYSCALL_INTERRUPT_PRIORITY when the critical section is entered, and 0 when the critical section is exited. Many bug reports are received that claim BASEPRI should be returned to its original value on exit, and not just set to zero, but the Cortex-M NVIC will never accept an interrupt that has a priority below that of the currently executing interrupt - no matter what BASEPRI is set to. An implementation that always sets BASEPRI to zero will result in faster code execution than an implementation that stores, then restores, the BASEPRI value (when the compiler's optimiser is turned on).

Relevance to the RTOS kernel

The RTOS kernel creates a critical section by writing the configMAX_SYSCALL_INTERRUPT_PRIORITY value into the ARM Cortex-M BASEPRI register. As priority 0 interrupts (the highest priority possible) cannot be masked using BASEPRI, configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to 0.